



**Bharat Ratna Indira Gandhi College of
Engineering, Solapur**
Master of computer application

**Subject: Data Structure and Algorithms
(MCAC102)**

Semester-I

Unit 1: Introduction to Data Structure

❖ Unit 01: Introduction to Data Structures

- **Data Structures** is about how data can be stored in different structures.
- **Algorithms** is about how to solve different problems, often by searching through and manipulating data structures.

Q. What are Data Structures?

- A data structure is a way to store data.
 - We structure data in different ways depending on what data we have, and what we want to do with it.
 - Data structures give us the possibility to manage large amounts of data efficiently for uses such as large databases and internet indexing services.
 - A data structure is not only used for organizing the data. It is also used for processing, retrieving, and storing data. Different basic and advanced types of data structures are used in almost every program or software system that has been developed. So, we must have good knowledge of data structures.
-

1. Data, Data Types, and Data Structures

- **Data:** Data refers to raw facts, figures, or statistics that are collected and stored for reference or analysis. It lacks context or meaning until processed.
 - **Example:** The number "5" by itself is data, but "5 units sold for \$100 each" gives it context, making it information.

- **Data Types:** Classifications that dictate the type of operations that can be performed on the data. These are typically classified as:
 - **Primitive Data Types:**
 - **Integer:** Whole numbers.
 - *Example:* `int x = 10;`
 - **Float:** Numbers with decimals.
 - *Example:* `float pi = 3.14;`
 - **Character:** Single letters or symbols.
 - *Example:* `char letter = 'A';`
 - **Boolean:** Represents True or False.
 - *Example:* `bool isStudent = true;`
 - **Composite Data Types:** These are built using primitive data types.
 - *Example:* Arrays (`int [] numbers = {1, 2, 3};`) or Structures (`struct Student {int id; char name [20];}`).
 - **Abstract Data Types (ADT):** These are models that define data structures by their operations without specifying how they are implemented.
 - *Example:* The Stack ADT allows operations like `push ()`, `pop ()`, and `peek ()` without detailing how the stack is stored in memory.

Data Types	Data Structures
1) Data Type is the kind or form of a variable which is being used throughout the program. It defines that the particular variable will assign the values of the given data type only	1) Data Structures is the collection of different kinds of data. That entire data can be represented using an object and can be used throughout the entire program.
2) Implementation through Data Types is a form of abstract implementation	2) Implementation through Data Structures is called concrete implementation
3) Can hold values and not data, so it is data less	3) Can hold different kind and types of data within one single object
4) No problem of time complexity	4) Time complexity comes into play when working with data structures
5) Examples: int, float, double	5) Examples: stacks, queues, tree

2. Data Structures

A **data structure** is a specialized way of organizing, managing, and storing data to make access and modification more efficient. They provide a framework for efficiently performing operations on data.

➤ Linear Data Structures:

- Data is arranged sequentially, and each element is connected to its previous and next element.
- *Example:* An **Array** stores elements contiguously in memory, e.g., `int arr[5] = {1, 2, 3, 4, 5};`.

❖ **Characteristics of Linear Data Structure:**

- I. **Sequential Organization:** In linear data structures, data elements are arranged sequentially, one after the other. Each element has a unique predecessor (except for the first element) and a unique successor (except for the last element)
- II. **Order Preservation:** The order in which elements are added to the data structure is preserved. This means that the first element added will be the first one to be accessed or removed, and the last element added will be the last one to be accessed or removed.
- III. **Fixed or Dynamic Size:** Linear data structures can have either fixed or dynamic sizes. Arrays typically have a fixed size when they are created, while other structures like linked lists, stacks, and queues can dynamically grow or shrink as elements are added or removed.

- IV. **Efficient Access:** Accessing elements within a linear data structure is typically efficient.

For example, arrays offer constant-time access to elements using their index.

➤ **Non-linear Data Structures:**

In **Data Structures and Algorithms (DSA)**, **non-linear data structures** are those where elements are not arranged in a sequential manner, meaning there is no clear linear relationship between consecutive elements. Instead, elements are organized in a hierarchical or interconnected way.

Non-linear structures allow for more complex relationships between elements, offering advantages for certain types of data and operations.

Example: A **Tree** where each node may have multiple children.

❖ **Characteristics of Non-Linear Data Structure:**

I. **Hierarchical Data Organization:**

- a. Non-linear data structures organize data in a hierarchical fashion, where elements are connected in a parent-child relationship.
- b. Examples include trees and graphs. In a tree, the top element is known as the root, and all other elements have a relationship as parent and children.

II. **Multiple Connections:**

- a. Each element in a non-linear data structure can connect to multiple elements, leading to complex relationships.

DSA-Unit [1]

- b. In a graph, for example, a node can have multiple edges connecting to other nodes, forming intricate paths.

III. Efficient Representation of Complex Data:

- a. Non-linear structures are more suited for representing complex relationships between data, such as hierarchical or interconnected data.
- b. Examples: Organizational hierarchies, social networks, web pages (represented as graphs).

IV. Traversal Methods:

- a. Traversing a non-linear data structure requires more sophisticated algorithms like Depth-First Search (DFS) or Breadth-First Search (BFS) for graphs and trees.
- b. This is different from linear traversal (e.g., left-to-right or right-to-left) used in linear data structures.

V. Memory Utilization:

- a. Non-linear data structures can be dynamic in nature and are typically stored in memory in a non-contiguous manner.
- b. For instance, a tree or graph uses pointers to link nodes, and each node could be placed at various locations in memory.

VI. Complexity in Operations:

- a. Operations like insertion, deletion, searching, and traversal are more complex compared to linear data structures due to the hierarchical relationships.
- b. For example, finding a node in a binary search tree requires navigating through multiple nodes based on key comparisons.

VII. Flexible Size:

- a. Non-linear data structures can grow and shrink dynamically, allowing flexibility in size.
- b. For example, a tree or graph can have an arbitrary number of nodes depending on the application's needs.

VIII. Types of Non-Linear Data Structures:

- The common types of non-linear data structures include:
 - Trees: Binary Tree, Binary Search Tree, AVL Tree, B-tree, etc.
 - Graphs: Directed Graph, Undirected Graph, Weighted Graph, etc.
 - Heaps: Max Heap, Min Heap.
 - Sets: Data structures representing a collection of distinct elements.

Examples:

- Tree: Used in hierarchical data like file systems or organizational charts.
 - **Graph: Used in network routing, social networks, or web page ranking systems.**
-

➤ **Abstract Data Type (ADT)**

An **ADT** defines the behavior of a data type through its operations, without concerning how it will be implemented.

- *Example:* A **Queue ADT** defines operations like enqueue () to add an element and dequeue () to remove one, without specifying the actual implementation (e.g., whether the queue is implemented using an array or linked list).

- **Domain:** Refers to the set of values the ADT can handle. In a **Queue**, for instance, the domain might be any collection of items, such as integers or strings.

An **Abstract Data Type (ADT)** is a theoretical concept used in data structures and algorithms to define data types abstractly, without specifying how they are implemented. ADTs define the behavior and operations on data, while the underlying implementation details are hidden. It focuses on **what** operations are performed, not **how** they are performed.

❖ Key Features of Abstract Data Types:

❖ Encapsulation:

- ADTs encapsulate both data and the operations that can be performed on the data.
- Users of an ADT do not need to know the internal workings or structure of the data; they interact only with the provided operations.

❖ Separation of Interface and Implementation:

- The **interface** of the ADT specifies the operations (functions or methods) available to users. It does not reveal how data is stored or how the operations are carried out.
- The **implementation** is the actual code or logic used to perform the operations defined by the ADT interface.
- This allows flexibility in changing the implementation without affecting the way the ADT is used.

❖ **Modularity:**

- ADTs promote modularity by separating concerns. The internal representation of data and the algorithms used can be changed without affecting the rest of the program.

❖ **Data Hiding:**

- ADTs hide the internal details of the data structure from the user, ensuring that the data is only accessible through the defined operations. This provides a layer of abstraction and increases security.

❖ **Improved Maintainability and Flexibility:**

- Since the ADT only exposes its interface, the implementation can be modified or optimized later without affecting the user code, improving the maintainability and flexibility of software.

❖ **Common ADTs**

I. List ADT:

- a. A list allows storing an ordered collection of elements with operations like insertion, deletion, and access by index.
- b. Operations: insert(), delete(), get(), size().

II. Stack ADT:

DSA-Unit [1]

- a. A stack is a Last In, First Out (LIFO) structure where elements are added and removed from one end, called the "top".
- b. Operations: push(), pop(), peek(), isEmpty().

III. Queue ADT:

- a. A queue is a First In, First Out (FIFO) structure where elements are added at the rear and removed from the front.
- b. Operations: enqueue(), dequeue(), peek(), isEmpty().

IV. Tree ADT:

- a. A tree organizes data hierarchically, with nodes connected by edges, starting from a root node.
- b. Operations: insert(), delete(), traverse().

V. Graph ADT:

- a. A graph consists of vertices (nodes) connected by edges (arcs), representing networks of relationships.
- b. Operations: addVertex(), addEdge(), getNeighbors().

VI. Dictionary ADT (Map):

- a. A dictionary stores key-value pairs, where each key is unique and maps to a specific value.
- b. Operations: put(key, value), get(key), remove(key), containsKey(key).

VII. Set ADT:

- A set is a collection of unique elements.
- Operations: add(element), remove(element), contains(element), union(), intersection().

❖ **Example of Stack ADT:**

- **Interface:**

- push(item): Adds an item to the top of the stack.
- pop(): Removes and returns the top item from the stack.
- peek(): Returns the top item without removing it.
- isEmpty(): Checks if the stack is empty.

- **Implementation** (using an array):

```
class Stack {  
  
    constructor() {  
  
        this.stack = [];  
  
    }  
  
    push(item) {  
  
        this.stack.push(item);  
  
    }  
  
    pop() {  
  
        if (!this.isEmpty()) {  
  
            return this.stack.pop();  
  
        }  
  
    }  
  
}
```

DSA-Unit [1]

```
        } else {  
            return null;  
        }  
    }  
  
    peek() {  
        if (!this.isEmpty()) {  
            return this.stack[this.stack.length - 1];  
        } else {  
            return null;  
        }  
    }  
  
    isEmpty() {  
        return this.stack.length === 0;  
    }  
}
```

❖ Importance of ADTs

- I. **Abstraction:** ADTs allow programmers to focus on high-level design by abstracting low-level implementation details.
- II. **Reusability:** Once an ADT is implemented, it can be reused in multiple applications without modifying the code.
- III. **Efficiency:** ADTs help optimize performance by allowing developers to choose different underlying implementations (e.g., arrays vs. linked lists for stacks).

- IV. **Error Reduction:** By encapsulating data and operations, ADTs reduce the chances of accidental data corruption and improve code reliability.
-

4. Representation of Information

Representation of Information in Data Structures

The representation of information refers to how data is structured in memory and organized within data structures. Efficient representation is key to optimizing performance and storage use. Below are common methods for representing information:

I. Primitive Data Types:

- a. Basic building blocks used for representing simple information like integers, floats, characters, and booleans.
- b. Example: int, float, char, boolean.

II. Arrays:

- a. A collection of elements (of the same type) stored in contiguous memory locations, accessible by index.
- b. Arrays provide sequential organization, allowing quick access but fixed size.
- c. Example: arr[5] stores 5 integers.

III. Linked Lists:

- a. A dynamic structure where elements (nodes) are stored in non-contiguous memory. Each node holds data and a reference (pointer) to the next node.
- b. Flexible in size, easy insertion and deletion but requires sequential access.
- c. Example: A list of nodes with each node pointing to the next.

IV. Stacks and Queues:

- a. Stack: Represents information in LIFO (Last In, First Out) order. Elements are added and removed from one end, called the "top."
- b. Queue: Represents information in FIFO (First In, First Out) order. Elements are added at the rear and removed from the front.

V. Trees:

- a. A hierarchical data structure where information is stored in nodes connected by edges. Each node has a parent-child relationship.
- b. Example: Binary Tree, where each node has at most two children.

VI. Graphs:

- a. A collection of nodes (vertices) connected by edges, used to represent relationships between data points in a network.
- b. Example: Social networks, computer networks.

VII. Hash Tables:

DSA-Unit [1]

- A structure that maps keys to values, allowing fast data access based on a unique key.
- Example: Dictionary in Python.

These representations are used based on the requirements of applications to optimize time and space complexity for various operations like insertion, deletion, and traversal.

Information representation involves storing data in ways that are understood by machines.

Common techniques include **binary encoding** and **ASCII**.

- *Example:* The character 'A' is represented as 01000001 in binary under the ASCII encoding scheme.
-

5. Characteristics of an Algorithm

An **algorithm** is a step-by-step process to solve a problem. It should possess certain characteristics:

- **Finiteness:** The algorithm must terminate after a finite number of steps.
 - *Example:* A **sorting algorithm** like Bubble Sort terminates once the list is sorted.
- **Definiteness:** Every step of the algorithm must be clearly defined.
 - *Example:* In Bubble Sort, the instruction "compare adjacent elements" is clear and well-defined.
- **Input/Output:** Algorithms take input and produce meaningful output.

- *Example:* **Input:** [4, 2, 1]; **Output:** [1, 2, 4] (sorted array).
 - **Effectiveness:** Each step must be executable and simple.
 - *Example:* Finding the maximum value in an array is a simple iteration.
-

6. Analyzing Programs

- I. **Time Complexity:** Describes how the running time of an algorithm grows with the input size.
 - *Example:* The time complexity of **Binary Search** is $O(\log n)$ because it divides the input size in half at each step.
 - II. **Space Complexity:** Describes how much memory an algorithm uses.
 - *Example:* A recursive function has higher space complexity due to maintaining the call stack.
-

7. Arrays and Hash Tables:

[Concept of Sequential Organization in Data Structures and Algorithms]

Sequential Organization in data structures refers to the way data is stored in a **contiguous block of memory**. It implies that the elements are placed one after the other, and each element is assigned a unique index or key, which can be used to access the element directly. The sequential

DSA-Unit [1]

nature allows for fast access, but operations like insertion and deletion may require shifting of elements, making these operations less efficient.

A. Arrays: Concept of Sequential Organization

An **array** is a collection of elements (typically of the same data type) stored at contiguous memory locations. The sequential organization in arrays allows for **direct access** to any element using its index. This makes arrays efficient for operations where the position of the element is known, such as random access or reading data at a specific index.

❖ Characteristics of Arrays:

- I. **Fixed Size:** Arrays have a fixed size defined at the time of declaration.
- II. **Contiguous Memory Allocation:** All elements are stored next to each other in memory, making access faster.
- III. **Direct Access via Indexing:** Any element can be accessed in constant time, i.e., $O(1)$, using its index.
- IV. **Time Complexity for Access:** $O(1)$ for direct access using the index.
- V. **Time Complexity for Insertion/Deletion:** $O(n)$ for insertion or deletion in the middle of the array, as shifting of elements is required.

Example:

Consider an integer array of size 5:

```
int arr[5] = {10, 20, 30, 40, 50};
```

DSA-Unit [1]

The elements are stored sequentially in memory:

Index: 0 1 2 3 4

Element: 10 20 30 40 50

To access the third element (`arr[2]`), you use the index 2:

```
int x = arr[2]; // x = 30
```

1. Memory Address Calculation for 1D Array

In a 1D array, all elements are stored in a single linear sequence.

Formula:

$$\text{Address}(A[i]) = \text{Base Address}(A) + i \times \text{Element Size}$$

Where:

- $A[i]$ is the element at index i
- $\text{Base Address}(A)$ is the memory address of the first element $A[0]$,
- i is the index of the element,
- Element Size is the size of each element in bytes.

Example: Let's assume:

- The base address of the array A is 1000,

DSA-Unit [1]

- The array stores integers (4 bytes each),
- We want to find the address of A[3].

Using the formula:

$$\text{Address}(A[3]) = 1000 + 3 \times 4 = 1000 + 12 = 1012$$

So, the address of A[3] is 1012.

2. Memory Address Calculation for 2D Array

A 2D array is essentially a matrix with rows and columns. There are two common ways to store 2D arrays in memory:

1. **Row-major order** (row by row),
2. **Column-major order** (column by column).

Row-major Order

In row-major order, elements are stored row by row.

Formula (Row-major):

$$\text{Address}(A[i][j]) = \text{Base Address}(A) + (i \times \text{Number of Columns} + j) \times \text{Element Size}$$

Where:

- i is the row index,
- j is the column index,

DSA-Unit [1]

- Number of Columns is the total number of columns in the 2D array.

Example: Let's assume:

- The base address of the array A is 2000,
- The array is a 3×4matrix,
- The array stores integers (4 bytes each),
- We want to find the address of A[2][1].

Using the formula:

$$\text{Address}(A[2][1]) = 2000 + (2 \times 4 + 1) \times 4 = 2000 + 9 \times 4 = 2000 + 36 = 2036$$

So, the address of A[2][1] is 2036.

Column-major Order

In column-major order, elements are stored column by column.

Formula (Column-major):

$$\text{Address}(A[i][j]) = \text{Base Address}(A) + (j \times \text{Number of Rows} + i) \times \text{Element Size}$$

Where:

- i is the row index,
- j is the column index
- Number of Rows is the total number of rows in the 2D array.

DSA-Unit [1]

Example: For the same array, in column-major order:

$$\text{Address}(A[2][1]) = 2000 + (1 \times 3 + 2) \times 4 = 2000 + 5 \times 4 = 2000 + 20 = 2020$$

So, the address of $A[2][1]$ in column-major order is 2020.

3. Memory Address Calculation for Multi-Dimensional Array (3D Array Example)

For higher-dimensional arrays (like 3D arrays), the memory layout becomes a bit more complex.

For a 3D array, you have three indices: i , j , and k , corresponding to the depth, row, and column respectively.

Row-major Order (3D Array)

In row-major order, elements are stored depth by depth, row by row.

Formula:

$$\text{Address}(A[i][j][k]) = \text{Base Address}(A) + (i \times (\text{Number of Rows} \times \text{Number of Columns}) + j \times \text{Number of Columns} + k) \times \text{Element Size}$$

Where:

- i is the depth (3rd index),
- j is the row (2nd index),
- k is the column (1st index).

DSA-Unit [1]

Example: Let's assume:

- The base address of the 3D array A is 3000,
- The array is 2×3×4 (2 depths, 3 rows, 4 columns),
- The array stores integers (4 bytes each),
- We want to find the address of A[1][2][3].

Using the formula:

$$\begin{aligned}\text{Address}(A[1][2][3]) &= 3000 + (1 \times (3 \times 4) + 2 \times 4 + 3) \times 4 \\ &= 3000 + (1 \times 12 + 8 + 3) \times 4 \\ &= 3000 + 23 \times 4 = 3000 + 92 \\ &= 3092\end{aligned}$$

So, the address of A[1][2][3] is 3092

B. Hash Tables: Concept of Sequential Organization

A **Hash Table** is a data structure that stores key-value pairs. It uses a **hash function** to compute an index (or hash code) for each key, which determines where the corresponding value is stored. Unlike arrays, hash tables don't require sequential memory allocation but offer an efficient way of organizing data for **fast access** via keys.

❖ **Characteristics of Hash Tables:**

- I. **Key-Value Mapping:** Each element in a hash table is stored as a key-value pair.
- II. **Hash Function:** A hash function converts the key into an index in the table, allowing for efficient access.
- III. **Non-Sequential Storage:** Unlike arrays, the data in hash tables is not stored sequentially in memory; it's stored based on the hash index.
- IV. **Collision Handling:** Since multiple keys can produce the same hash code (collision), techniques like **chaining** or **open addressing** are used to resolve them.

Example:

Consider a hash table to store employee names based on their employee ID as a key:

HashTable ht;

ht.put(101, "Alice");

ht.put(102, "Bob");

ht.put(103, "Charlie");

A hash function, say $h(\text{key}) = \text{key} \% 10$, generates the index:

$h(101) = 1$, $h(102) = 2$, $h(103) = 3$.

The hash table is conceptually organized like this:

DSA-Unit [1]

Index: 0 1 2 3

Element: 101->Alice 102->Bob 103->Charlie

To access an element:

```
ht.get(102); // Returns "Bob"
```

- **Time Complexity for Access:** Ideally $O(1)$ with a good hash function.
- **Time Complexity for Insertion/Deletion:** $O(1)$ on average.

Collision Handling Example (Chaining):

If two keys map to the same index, e.g., $h(111) = h(101) = 1$, a **linked list** (or another structure) at that index can store both elements:

Index 1: 101->Alice -> 111->Eve

❖ Key Differences Between Arrays and Hash Tables

Feature	Array	Hash Table
Data Access	Direct access via index ($O(1)$)	Access via key using hash function ($O(1)$ average)
Storage Type	Sequential, contiguous memory	Non-sequential, based on hash function

Feature	Array	Hash Table
Insertion/Deletion	Slow in the middle ($O(n)$)	Fast ($O(1)$ on average)
Fixed Size	Yes, declared at initialization	No, dynamic size (can grow or shrink)
Collision Handling	Not applicable	Requires handling (e.g., chaining)

8. Sequential Organization

In **sequential organization**, data is stored one after the other in memory or disk.

- *Example:* In a **file system**, files are often written sequentially to disk.
-

9. Linear and Non-linear Data Structures

- **Linear Data Structures:** Data elements are arranged sequentially.
 - *Example:* A **Stack** follows the **LIFO** principle (Last In, First Out). It's like a stack of plates where the last plate added is the first to be removed.
 - A **Queue** follows the **FIFO** principle (First In, First Out). It's like a queue in a store, where the first person in line is served first.
- **Non-linear Data Structures:** Data elements are stored in a hierarchical way.

DSA-Unit [1]

- *Example:* A **Binary Tree** has a root node with left and right children, forming a branching structure.
-

10. Storage Representation

Efficient storage management ensures that data is stored and accessed in the least time-consuming manner.

- *Example:* **Pointers** are used in linked lists to store the memory addresses of the next element.
-

11. Array Processing

Array processing involves various operations like traversing, searching, and sorting.

- *Example:* **Sorting** an array using **Selection Sort**.
 - **Input:** [64, 25, 12, 22, 11]
 - **Output after sorting:** [11, 12, 22, 25, 64]
-

12. Sparse Matrices

A **Sparse Matrix** is a matrix in which most of the elements are zero. To save memory, only non-zero elements are stored along with their positions.

DSA-Unit [1]

- *Example:*

[0, 0, 0, 0]

[0, 0, 5, 0]

[0, 0, 0, 8]

[0, 0, 0, 0]

Instead of storing all values, only the non-zero elements (5 at (1,2)) and (8 at (2,3)) are stored.

A **sparse matrix** is a matrix in which the majority of its elements are zero (or any default value).

In such matrices, non-zero elements are few compared to the total number of elements. **Sparse matrices** often arise in areas like machine learning, scientific computing, graph theory, and network analysis, where data is highly scattered, and most of the matrix elements are not meaningful. Storing and processing sparse matrices efficiently is crucial since storing all the zeros (or default values) would be a waste of both memory and computational resources.

❖ **Characteristics of Sparse Matrices:**

- I. **High Percentage of Zeros:** Sparse matrices contain a large number of zero (or default) elements.
- II. **Efficient Storage:** Since most elements are zero, only the non-zero elements need to be stored, reducing memory usage.
- III. **Specialized Algorithms:** Sparse matrix operations like multiplication, addition, and transposition require algorithms that only focus on the non-zero elements to optimize performance.

❖ **Types of Sparse Matrices:**

I. **Diagonal Matrix:**

a. Only the diagonal elements (from top-left to bottom-right) are non-zero. All other elements are zero.

b. Example:

i. [5, 0, 0]

ii. [0, 3, 0]

iii. [0, 0, 7]

II. **Triangular Matrix:**

a. Either the upper triangular part or the lower triangular part of the matrix contains non-zero elements, while the rest are zeros.

b. Example (Lower Triangular):

i. [1, 0, 0]

ii. [2, 3, 0]

iii. [4, 5, 6]

III. **Band Matrix:**

○ Non-zero elements are located near the diagonal, forming a band-like structure.

The rest of the matrix elements are zeros.

○ Example (with a bandwidth of 1):

DSA-Unit [1]

[3, 2, 0, 0]

[1, 5, 3, 0]

[0, 6, 8, 4]

[0, 0, 7, 9]

13. Transpose of Sparse Matrices

The **transpose** of a matrix is obtained by switching the rows and columns of the matrix.

- *Example:* If the original matrix has an element at position (1, 2), its transposed version will have the element at position (2, 1).

The **transpose of a matrix** is a new matrix obtained by flipping the matrix over its diagonal, which essentially switches the row and column indices of each element. For a sparse matrix, the goal is to perform the transpose efficiently without wasting memory or computational resources on the zero elements. The sparse matrix transpose is widely used in applications like graph algorithms, where adjacency matrices are transposed, and in other fields requiring matrix operations.

Definition:

If matrix **A** is an $m \times n$ matrix, its transpose matrix **A^T** is an $n \times m$ matrix where the elements are:

$$A^T[i][j] = A[j][i]$$

Representation of Sparse Matrices for Transposition

DSA-Unit [1]

Sparse matrices can be stored using various compact representations such as **Coordinate List (COO)**, **Compressed Sparse Row (CSR)**, or **Compressed Sparse Column (CSC)**. The transposition of a sparse matrix in any of these formats only involves adjusting the row and column indices of the non-zero elements.

A. Transpose in Coordinate List (COO) Representation

In the **COO representation**, a sparse matrix is stored as a list of tuples where each tuple represents the row, column, and value of the non-zero elements.

Example:

Given the matrix **A**:

$A = \begin{bmatrix} 0 & 0 & 3 \\ 0 & 4 & 0 \\ 5 & 0 & 0 \end{bmatrix}$

The COO representation of matrix **A** is:

COO Representation of A:

(Row, Column, Value)

(0, 2, 3)

(1, 1, 4)

(2, 0, 5)

The transpose **A^T** is:

$A^T = \begin{bmatrix} 0 & 0 & 5 \\ 0 & 4 & 0 \end{bmatrix}$

DSA-Unit [1]

[3, 0, 0]]

To transpose in COO, simply swap the row and column indices:

COO Representation of A^T :

(Row, Column, Value)

(2, 0, 3)

(1, 1, 4)

(0, 2, 5)

B. Transpose in Compressed Sparse Row (CSR) Representation

The **CSR representation** stores sparse matrices using three arrays:

- I. **Values:** Non-zero elements.
- II. **Column Indices:** The column index of each non-zero element.
- III. **Row Pointers:** Points to the starting index of each row in the values array.

Example:

Matrix **A**:

$A = \begin{bmatrix} 0 & 0 & 3 \\ 0 & 4 & 0 \\ 5 & 0 & 0 \end{bmatrix}$

CSR Representation of **A**:

- **Values:** [3, 4, 5]
- **Column Indices:** [2, 1, 0]

DSA-Unit [1]

- **Row Pointers:** [0, 1, 2, 3]

To transpose the matrix in CSR format:

1. Create a new **Values** array that will store the non-zero elements in the transposed order.
2. Swap the roles of the **Column Indices** and **Row Pointers** to reflect the row-major order of the transposed matrix.

CSR Representation of A^T :

- **Values:** [5, 4, 3]
- **Column Indices:** [2, 1, 0]
- **Row Pointers:** [0, 1, 2, 3]

The CSR transpose involves modifying the internal structure by reordering the column indices and row pointers.

C. Transpose in Compressed Sparse Column (CSC) Representation

The **CSC representation** is similar to CSR but is column-oriented. To perform the transpose of a matrix in CSC format:

- Swap the row indices and column pointers.

Example:

Given matrix **A** in **CSC** format:

- **Values:** [5, 4, 3]
- **Row Indices:** [2, 1, 0]
- **Column Pointers:** [0, 1, 2, 3]

The transpose A^T in **CSC** format would swap the roles of row indices and column pointers.

Algorithm for Transposing a Sparse Matrix

The basic algorithm to transpose a sparse matrix focuses on efficiently rearranging the non-zero elements. Here's a generalized approach to transpose a sparse matrix in any format.

❖ Steps for Transposing a Sparse Matrix:

- I. **Iterate through the non-zero elements:** For each non-zero element (i, j, value) in the original matrix, move it to position (j, i, value) in the transposed matrix.
- II. **Create new storage:** For the transposed matrix, initialize new arrays to store the non-zero elements along with their new row and column positions.
- III. **Adjust pointers (if necessary):** If using a compressed representation like CSR or CSC, adjust the row/column pointers to reflect the new positions of the elements.

❖ Applications of Sparse Matrix Transpose

- I. **Graph Algorithms:** When representing graphs as adjacency matrices, transposing the matrix is equivalent to reversing the direction of all edges in the graph.
- II. **Parallel Computing:** Transposing matrices is useful in distributed computing environments, where matrix operations are distributed across processors.
- III. **Machine Learning:** In many algorithms (e.g., matrix factorization), the transpose of a sparse matrix is frequently used to simplify or optimize computations.

Example Problem: Transposing a Sparse Matrix

DSA-Unit [1]

Consider the following sparse matrix **A**:

$$A = \begin{bmatrix} 0 & 0 & 3 & 0 \\ 0 & 4 & 0 & 0 \\ 5 & 0 & 0 & 7 \end{bmatrix}$$

The COO representation of **A** is:

$$(0, 2, 3)$$

$$(1, 1, 4)$$

$$(2, 0, 5)$$

$$(2, 3, 7)$$

Transpose of matrix **A** (**A^T**):

$$A^T = \begin{bmatrix} 0 & 0 & 5 \\ 0 & 4 & 0 \\ 3 & 0 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

COO Representation of **A^T**:

$$(2, 0, 3)$$

$$(1, 1, 4)$$

$$(0, 2, 5)$$

$$(3, 2, 7)$$

DSA-Unit [1]

1. Sparse Matrices:

A **sparse matrix** is a matrix in which most of the elements are zero. Instead of storing every element of the matrix (including the zeros), sparse matrix formats store only the non-zero elements and their positions to save memory.

2. Matrix Transpose:

The **transpose** of a matrix is obtained by swapping its rows and columns. For example, if A is an $m \times n$ matrix, then the transpose A^T is an $n \times m$ matrix where:

$$A^T[i,j] = A[j,i]$$

This operation swaps the rows and columns of the matrix.

3. COO Format (Coordinate List):

In **COO** format, the sparse matrix is stored as a list of (row, column, value) tuples, where:

- row: The row index of the non-zero element.
- col: The column index of the non-zero element.
- data: The value of the non-zero element.

Transpose in COO Format:

To obtain the transpose of a sparse matrix in COO format:

1. Swap the row and col indices for each non-zero entry.
2. Keep the data (non-zero values) unchanged.

DSA-Unit [1]

Example: Consider the following matrix A:

$$A = \begin{pmatrix} 0 & 2 & 0 \\ 3 & 0 & 4 \\ 4 & 0 & 5 \end{pmatrix}$$

The COO representation of A is:

- row = [0, 1, 1, 2]
- col = [1, 0, 2, 2]
- data = [2, 3, 4, 5]

For A^T (transpose of A):

- Swap the row and col values:
row = [1, 0, 2, 2] col = [0, 1, 1, 2]
- data remains the same: [2, 3, 4, 5]

Thus, the transpose A^T is:

$$A^T = \begin{pmatrix} 0 & 3 & 0 \\ 2 & 0 & 0 \\ 0 & 4 & 5 \end{pmatrix}$$

4. CSR Format (Compressed Sparse Row):

In **CSR** format, the matrix is stored in three arrays:

- **data:** Stores the non-zero values.
- **indices:** Stores the column indices of the non-zero values.
- **indptr:** Stores the cumulative sum of non-zero elements in each row (i.e., it points to the start of each row in the data and indices arrays).

Transpose in CSR Format:

To transpose a matrix in CSR format:

1. **Convert the CSR matrix to COO format:** This allows you to easily swap rows and columns.
2. **Swap row and column indices:** Like in COO, swap the row and column indices to get the transpose.
3. **Rebuild CSR:** Finally, convert the transposed COO matrix back into CSR format.

Example (CSR Format):

Consider the same matrix A:

$$A = \begin{pmatrix} 0 & 2 & 0 \\ 3 & 0 & 4 \\ 0 & 0 & 5 \end{pmatrix}$$

DSA-Unit [1]

The CSR representation is:

- data = [2, 3, 4, 5]
- indices = [1, 0, 2, 2]
- indptr = [0, 1, 3, 4]

For ATA^TAT :

1. Convert CSR to COO:

- row = [0, 1, 1, 2] (corresponding to the row indices)
- col = [1, 0, 2, 2] (column indices)
- data = [2, 3, 4, 5]

2. Transpose in COO format: Swap the row and col:

- row = [1, 0, 2, 2]
- col = [0, 1, 1, 2]
- data = [2, 3, 4, 5]

3. Convert back to CSR:

- data = [3, 2, 4, 5]
- indices = [0, 1, 1, 2]
- indptr = [0, 1, 2, 4]

DSA-Unit [1]

$$A^T = \begin{pmatrix} 0 & 3 & 0 \\ 2 & 0 & 0 \\ 0 & 4 & 5 \end{pmatrix}$$

Summary:

- **COO Transpose:** Simply swap the row and column indices of the non-zero elements.
- **CSR Transpose:** Convert CSR to COO, swap row and column indices, and then convert back to CSR.